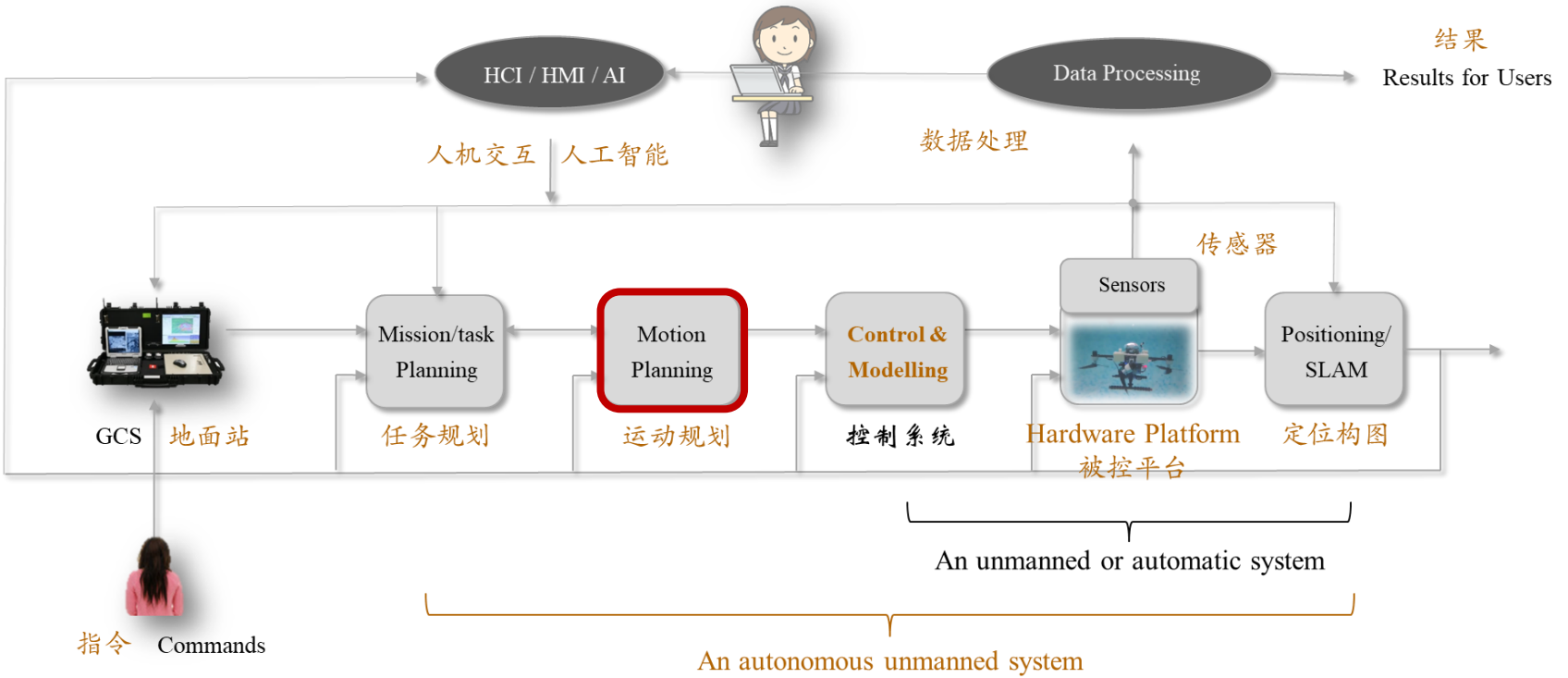


UAV Motion and Task Planning

© Copyright Unmanned Systems Research Group

Motion Planning



© Copyright Unmanned Systems Research Group



Outline

- Introduction
- Differential Flatness
- Polynomial Trajectory
- Minimum Snap Trajectory Generation
- Time Allocation
- Corrido-based Trajectory Optimization (Optional)

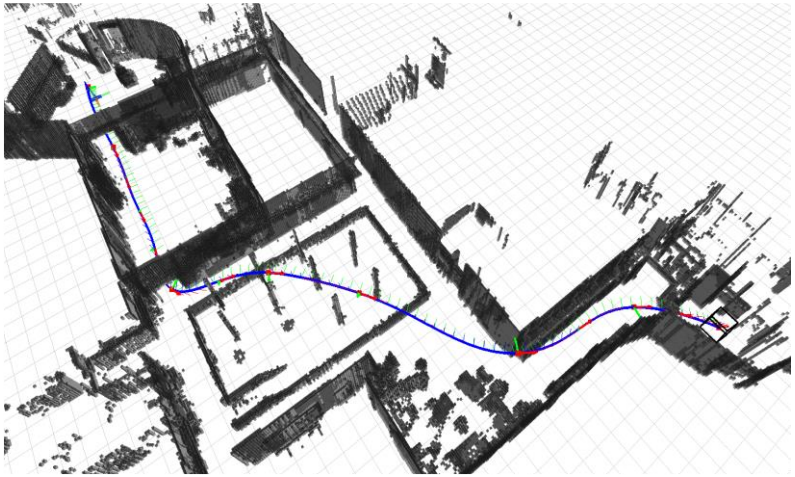
© Copyright Unmanned Systems Research Group

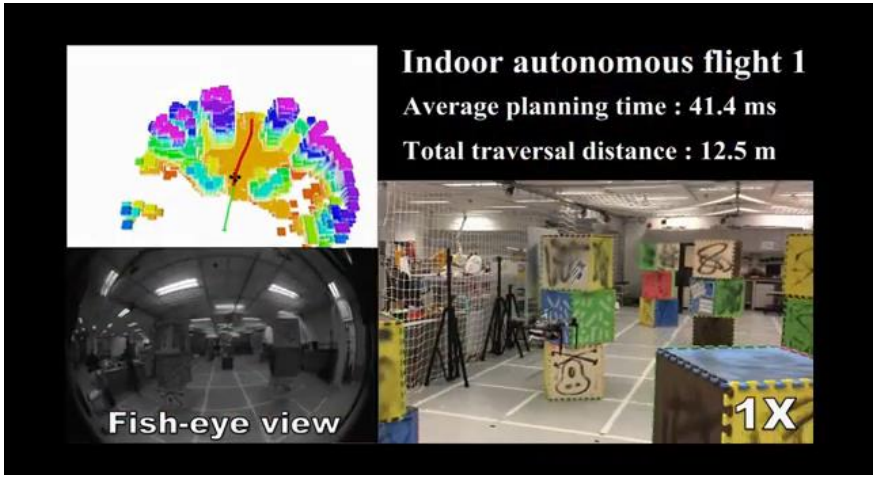
What are Trajectories?

Trajectory generation is a core component in UAV autonomy, enabling the planning of feasible, time-parameterized flight paths.

- Trajectories are time-parameterized paths
- Trajectories consist of spatial and temporal profiles

Difference between trajectory and path?



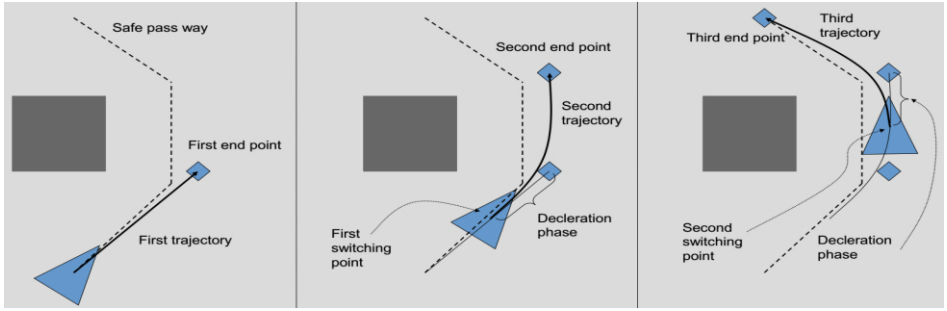


Richter C, BryA, Roy N. Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments[M]//Robotics research. Springer, Cham, 2016: 649-666.

© Copyright Unmanned Systems Research Group

Why to Use Trajectory Optimization

- **Dynamic Feasibility**: Satisfies kinematic and dynamic limits (e.g., speed, thrust, angular rates).
- **Safe and Smooth Navigation**: Ensures collision-free and safe flight in complex environments.
- **Precise Motion Planning**: Enables accurate tracking performance over time.
- **Planning-Control Integration**: Bridges high-level mission planning with low-level control execution.
- **Real-Time Adaptability**: Supports in-flight replanning in response to obstacles or disturbances (e.g., wind).
- **What else??**



© Copyright Unmanned Systems Research Group

Constraints for Trajectories

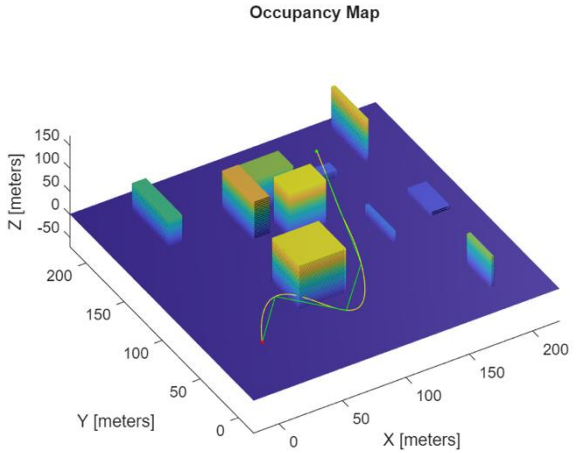
Constraints ensure that planned trajectories are feasible, safe, and executable. Effective constraint handling is key to realistic motion planning and successful trajectory optimization.

➢Kinematic Constraints

- Limit turning radius
- Typically modeled as inequality constraints in optimization

➢Dynamic Constraints

- Bound acceleration/deceleration due to motor limits
- Enforce jerk limits for smoother motion
- Often nonlinear in optimization formulations



➢Environmental Constraints

- Account for static (e.g., buildings) and dynamic (e.g., other drones) obstacles
- Implemented via collision avoidance and safety buffers

© Copyright Unmanned Systems Research Group

Smoothness and Continuity

Smooth and continuous trajectories are essential for safety, efficiency, and system longevity.

➢Why Smoothness Matters

- Reduces jerk and abrupt acceleration changes for onboard sensors, e.g. cameras.
- Lowers energy consumption by avoiding sharp maneuverers.
- Minimizes mechanical stress on actuators and components.
- Improves perceived reliability and quality of autonomous flight.

➢Role of Continuity

- **C^0 continuity**: Ensures position continuity (no jumps).
- **C^1 continuity**: Guarantees continuous velocity transitions.
- **C^2 continuity**: Provides continuous acceleration for safe, comfortable motion.
- **Higher-order (C^3, C^4)**: Continuous jerk and snap for fine control.

© Copyright Unmanned Systems Research Group

Common Trajectory Generation Methods

➢Polynomial-Based Approaches

- Use polynomials to generate smooth trajectories satisfying boundary conditions.
- Efficient, closed-form solutions; ideal for simple maneuvers with known endpoints.

➢Spline-Based Techniques

- Use piecewise polynomials (e.g., cubic splines, B-splines, Bézier curves) for smooth, flexible paths.
- Allow local control and are suitable for complex trajectories.

➢Optimization-Based Methods

- Formulate trajectory planning as an optimization problem.
- Methods like MPC handle constraints and multi-objective trade-offs.
- Suitable for dynamic, real-time applications.

© Copyright Unmanned Systems Research Group



Outline

- Introduction
- Differential Flatness
- Polynomial Trajectory
- Minimum Snap Trajectory Generation
- Time Allocation
- Corrido-based Trajectory Optimization (Optional)

© Copyright Unmanned Systems Research Group

Differential Flatness

What is Differential Flatness?

Differential flatness is not an algorithm, but a structural property of a system. For systems with this property, such as UAVs, ground vehicles, or robotic arms, complex trajectory planning and control problems can be transformed into **trajectory design problems in the output space**.

The states and the inputs of a quadrotor can be written as **algebraic functions of four carefully selected flat outputs and their derivatives**

- Enables automated generation of trajectories
- Any smooth trajectory in the space of **flat outputs** (with reasonably bounded derivatives) can be followed by the under actuated quadrotor
- A possible choice:

$$\boldsymbol{\sigma} = [p_x, p_y, p_z, \psi]^T$$

- Trajectory in the space of flat outputs:

$$\boldsymbol{\sigma}(t)\colon [T_0, T_M] \rightarrow \mathbb{R}^3 \times SO(2)$$

Minimum Snap Trajectory Generation and Control for Quadrotors, Daniel Mellinger and Vijay Kumar

© Copyright Unmanned Systems Research Group

Differential Flatness

Why to Use Differential Flatness

- Differential flatness is not an algorithm, but a structural property of a system.
- For systems with this property, such as UAVs, ground vehicles, or robotic arms, complex trajectory planning and control problems can be transformed into trajectory design problems in the output space. This represents a kind of 'decoupling miracle' in control system design.

© Copyright Unmanned Systems Research Group

Proof of Differential Flatness of Quadrotor

Quadrotor states

Position, orientation, linear velocity, angular velocity

$$\mathbf{X} = \left[p_x, p_y, p_z, \phi, \theta, \psi, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z\right]^T$$

Equation of motions:

$$m\ddot{\mathbf{p}} = mg\mathbf{e}_3 + u_1\mathbf{z}_\mathrm{b}.$$

$$\dot{\boldsymbol{\omega}}_\mathrm{b} = \mathbf{J}^{-1}[-\boldsymbol{\omega}_\mathrm{b}\times\mathbf{J}\cdot\boldsymbol{\omega}_\mathrm{b} + \boldsymbol{u}_2]$$

Position, velocity, and acceleration are simply derivatives of the flat outputs

Choose flat outputs:

$$\boldsymbol{\sigma} = \left[p_x, p_y, p_z, \psi\right]^T$$

Show all states and inputs can be expressed using the flat outputs and their derivatives

Minimum Snap Trajectory Generation and Control for Quadrotors, Daniel Mellinger and Vijay Kumar

© Copyright Unmanned Systems Research Group

Proof of Differential Flatness of Quadrotor

Computing Translational States from Flat Outputs

Position:

$$\mathbf{p}(t) = \left[p_x, p_y, p_z\right]^T$$

Velocity:

$$\mathbf{v}(t) = \dot{\mathbf{p}}(t)$$

Note:

The position, velocity, and acceleration of the center of mass correspond to the first, second, and third derivatives of the flat output, respectively.

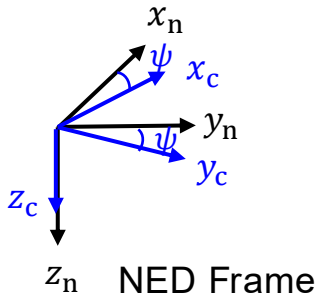
© Copyright Unmanned Systems Research Group

Proof of Differential Flatness of Quadrotor

Computing Thrust from Flat Outputs

From the translational dynamics

$$m\ddot{\mathbf{p}} = mg\mathbf{e}_3 + u_1\mathbf{z}_\mathrm{b} \implies u_1\mathbf{z}_\mathrm{b} = m(\ddot{\mathbf{p}} - g\mathbf{e}_3)$$

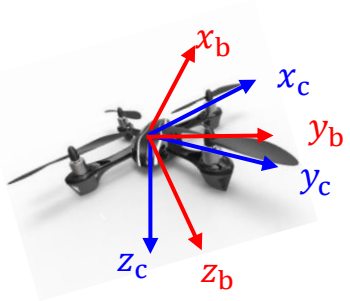


Then, we obtain

$$\mathbf{z}_\mathrm{b} = \frac{m(\ddot{\mathbf{p}} - g\mathbf{e}_3)}{u_1}$$

But by definition, $\mathbf{z}_\mathrm{b}$ is a unit vector (i.e., $\| \mathbf{z}_\mathrm{b} \| = 1$), representing the direction of the body's z-axis in world coordinates. So, take the norm of both sides:

$$1 = \left\| \frac{m(\ddot{\mathbf{p}} - g\mathbf{e}_3)}{u_1} \right\| \implies u_1 = m \cdot \|\ddot{\mathbf{p}} - g\mathbf{e}_3\|$$



Substituting this back into the expression:

$$\mathbf{z}_\mathrm{b} = \frac{m(\ddot{\mathbf{p}} - g\mathbf{e}_3)}{m \cdot \|\ddot{\mathbf{p}} - g\mathbf{e}_3\|} = \frac{\ddot{\mathbf{p}} - g\mathbf{e}_3}{\|\ddot{\mathbf{p}} - g\mathbf{e}_3\|}$$

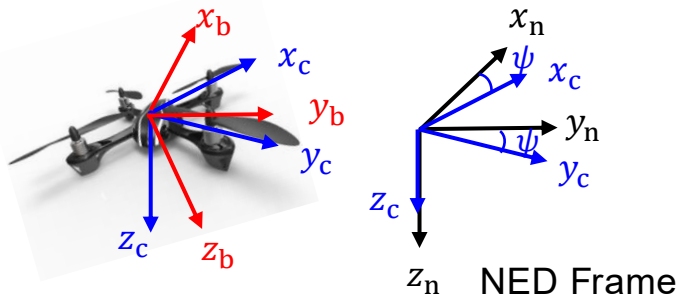
© Copyright Unmanned Systems Research Group

Proof of Differential Flatness of Quadrotor

Computing Orientation from Flat Outputs

Construct the rotation matrix $\mathbf{R} \in SO(3)$, with:

- $\mathbf{x}_b$: forward direction (body x-axis in NED frame)
- $\mathbf{y}_b$: lateral direction (body y-axis)
- $\mathbf{z}_b$: downward direction (already known)



The desired heading is the projection of the drone's x-axis onto the xy-plane, defined by rotating the world frame around the z-axis by a specified yaw angle. The corresponding unit vectors $\mathbf{x}_c$ and $\mathbf{y}_c$ in the NED frame are given by:

$$\mathbf{x}_c = [\cos\psi, \sin\psi, 0]^T \text{ and } \mathbf{y}_c = [-\sin\psi, \cos\psi, 0]^T$$

Intermediate coordination system

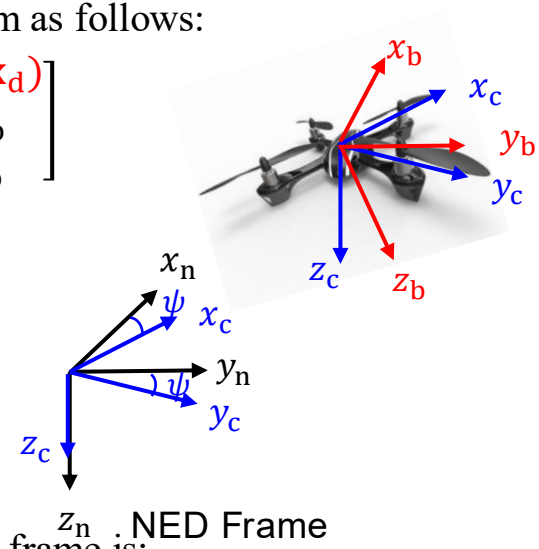
© Copyright Unmanned Systems Research Group

Proof of Differential Flatness of Quadrotor

Computing Orientation from Flat Outputs

Since we adopt the 'Z-Y-X' rotation sequence, the coordinate axes transform as follows:

$$\begin{bmatrix}\mathbf{x}_n\\ \mathbf{y}_n\\ \mathbf{z}_n\end{bmatrix} \xrightarrow{\text{Rotate } \mathbf{z}_n} \begin{bmatrix}\mathbf{x}_c\\ \mathbf{y}_c\\ \mathbf{z}_c(\mathbf{z}_n)\end{bmatrix} \xrightarrow{\text{Rotate } y_c} \begin{bmatrix}\mathbf{x}_d\\ \mathbf{y}_d(\mathbf{y}_c)\\ \mathbf{z}_d\end{bmatrix} \xrightarrow{\text{Rotate } x_d} \begin{bmatrix}\mathbf{x}_b(\mathbf{x}_d)\\ \mathbf{y}_b\\ \mathbf{z}_b\end{bmatrix}$$



Since $\mathbf{x}_b$ is orthogonal to $\mathbf{y}_c$ and $\mathbf{z}_b$, we compute

$$\mathbf{x}_b = \frac{\mathbf{y}_c \times \mathbf{z}_b}{\|\mathbf{y}_c \times \mathbf{z}_b\|}$$

Since $\boldsymbol{y}_b$ is orthogonal to $\mathbf{x}_b$ and $\mathbf{z}_b$, we compute

$$\mathbf{y}_b = \frac{\mathbf{z}_b \times \mathbf{x}_b}{\|\mathbf{z}_b \times \mathbf{x}_b\|}$$

The resulting rotation matrix representing the body orientation in the world frame is:

$$\mathbf{R} = [\mathbf{x}_b \ \ \mathbf{y}_b \ \ \mathbf{z}_b]$$

This matrix can be expressed in terms of flat outputs.

© Copyright Unmanned Systems Research Group

Differential Flatness of Quadrotor

Compute Angular Velocity from Flat Outputs

Angular velocity can be expressed in terms of flat outputs

$$\dot{\mathbf{R}} = \mathbf{R}\hat{\boldsymbol{\omega}}_{\mathrm{b}}, \qquad \boldsymbol{\omega}_{\mathrm{b}} = \left(\mathbf{R}^{T}\dot{\mathbf{R}}\right)^{\vee}$$

where $\hat{\boldsymbol{\omega}}_{\mathrm{b}}$ is the skew-symmetric matrix form of $\boldsymbol{\omega}_{\mathrm{b}}$, defined as

$$\hat{\boldsymbol{\omega}}_{\mathrm{b}} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

and $(\cdot)^{\vee}$ maps a skew-symmetric matrix to a vector.

$\boldsymbol{u}_2$ can be obtained according to:

$$\dot{\boldsymbol{\omega}}_{\mathrm{b}} = \mathbf{J}^{-1}[-\boldsymbol{\omega}_{\mathrm{b}} \times \mathbf{J} \cdot \boldsymbol{\omega}_{\mathrm{b}} + \boldsymbol{u}_2]$$

Finally, we have finished the derivation of flatness transformation

Planning flat-output trajectories with high order continuity suffices for the dynamics.

© Copyright Unmanned Systems Research Group

Differential Flatness of Quadrotor

Summary

- Quadrotor states

$$\mathbf{X} = [p_x, p_y, p_z, \phi, \theta, \psi, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$$

- Flat outputs:

$$\boldsymbol{\sigma} = [p_x, p_y, p_z, \psi]^T$$

- Position, velocity, and acceleration: Derivatives of flat outputs
- Orientation:

$$\mathbf{x}_c = [\cos\psi, \sin\psi, 0]^T \rightarrow \mathbf{R} = [\mathbf{x}_b \ \mathbf{y}_b \ \mathbf{z}_b]$$

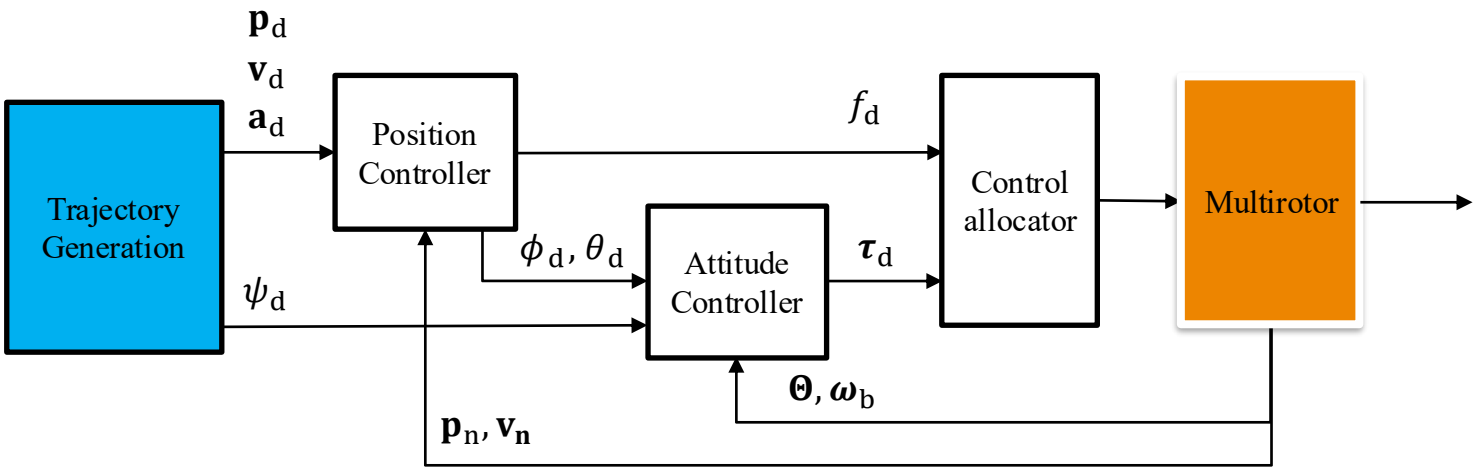
- Angular velocity

$$\dot{\boldsymbol{\omega}}_b = \mathbf{J}^{-1}[-\boldsymbol{\omega}_b \times \mathbf{J} \cdot \boldsymbol{\omega}_b + \boldsymbol{u}_2]$$

Done. You should only remember the planning of a UAV can be done in p_x, p_y, p_z, ψ.

© Copyright Unmanned Systems Research Group

Close the Trajectory Generation and Control Loop



© Copyright Unmanned Systems Research Group



Outline

- Introduction
- Differential Flatness
- Polynomial Trajectory
- Minimum Snap Trajectory Generation
- Closed form Solution to Minimum Snap
- Implementation Details

© Copyright Unmanned Systems Research Group

Polynomial Trajectories

- Flat outputs:

$$\boldsymbol{\sigma} = \left[p_x, p_y, p_z, \psi\right]^T$$

- Trajectory in the space of flat outputs:

$$\boldsymbol{\sigma}(t)\colon [T_0, T_M] \rightarrow \mathbb{R}^3 \times SO(2)$$

- Polynomial functions can be used to specify trajectories in the space of flat outputs
  - Easy determination of smoothness criterion with polynomial orders
  - Easy and closed form calculation of derivatives
  - Decoupled trajectory generation in three dimensions

© Copyright Unmanned Systems Research Group

Single-Segment 1D Trajectory

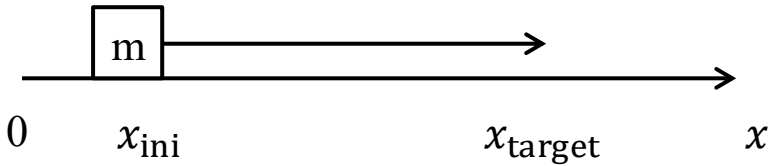
Example: Motion of a 1D Mass Block

How to represent the motion of a mass block moving from an initial position x_{ini} to a target position x_{target}?

This is a Boundary Value Problem (BVP)

Trajectory Representation:

$$x = f(t)$$



Boundary Conditions:

$$x_0 = f(0) = x_{\text{ini}}$$
$$x_f = f(t_f) = x_{\text{target}}$$

Polynomial functions are commonly used to represent motion trajectories.

© Copyright Unmanned Systems Research Group

Single-Segment 1D Trajectory

Trajectory as a Function of Time:

$$x = f(t) = at^2 + b\,t + c$$

Extension to Multiple Dimensions:

This formulation naturally extends to multi-dimensional motion:

$$x = f(t)$$
$$y = g(t)$$
$$z = h(t)$$
$$\ldots$$



Real-world execution of polynomial trajectory

© Copyright Unmanned Systems Research Group

Single-Segment 1D Trajectory

Solve for the polynomial coefficients that define the trajectory.

$x = f(t) = at^2 + bt + c$

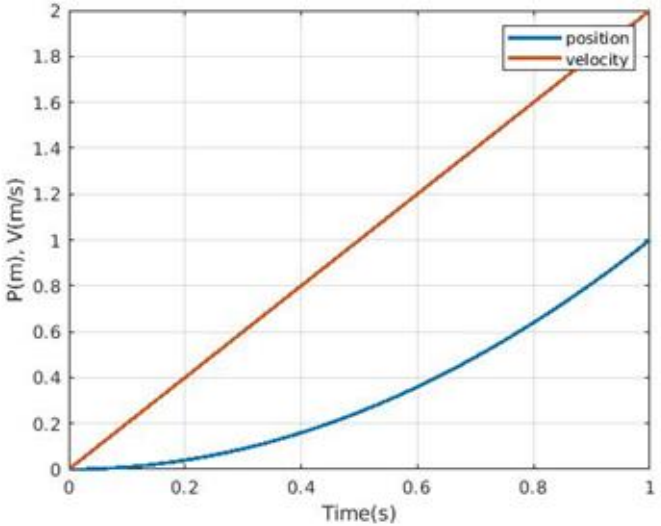
Trajectory Parametrization

Boundary Conditions:

$$f(0) = x_{\text{ini}}$$
$$f(1) = x_{\text{target}}$$

Parameter Calculations:

$$c = x_{\text{ini}}$$
$$a + b + c = x_{\text{target}}$$



Trajectory and velocity profile for 1D polynomial motion

© Copyright Unmanned Systems Research Group

Single-Segment 1D Trajectory

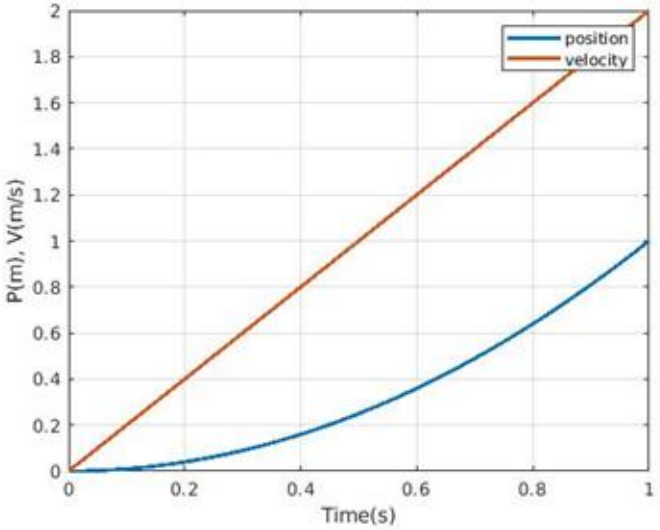
Adding a Velocity Constraint:

To refine the trajectory, an additional constraint can be introduced, for example, setting the initial velocity to zero:

$$v(t) = \frac{df}{dt} = 2at + b$$
$$v(0) = b = 0$$

Resulting System of Equations:

$$c = x_{\text{ini}}$$
$$a + b + c = x_{\text{target}}$$
$$b = 0$$



© Copyright Unmanned Systems Research Group

Single-Segment 1D Trajectory

The trajectory constraints can be represented in matrix form:

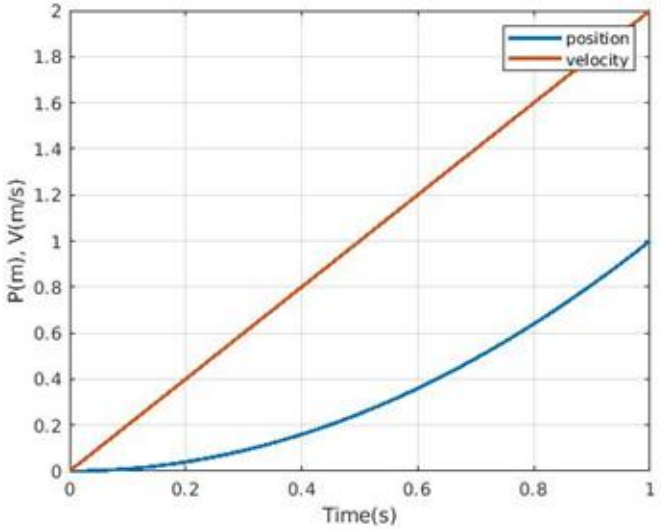
$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} x_{\text{ini}} \\ x_{\text{target}} \\ 0 \end{bmatrix}$$

This is of a typical linear programming problem, which can be written as

$$Ax = b$$

You can compute the coefficients using the backslash operator in MATLAB

$$x = A\backslash \text{b}$$



© Copyright Unmanned Systems Research Group

Single-Segment 1D Trajectory

Extending the Polynomial to Enforce Stop Conditions

What if we also want the **final velocity to be zero**, so that the mass block comes to rest at the target position?

Solution:

Increase the number of variables in the polynomial (by increasing the order of the polynomial)

Cubic Polynomial Trajectory:

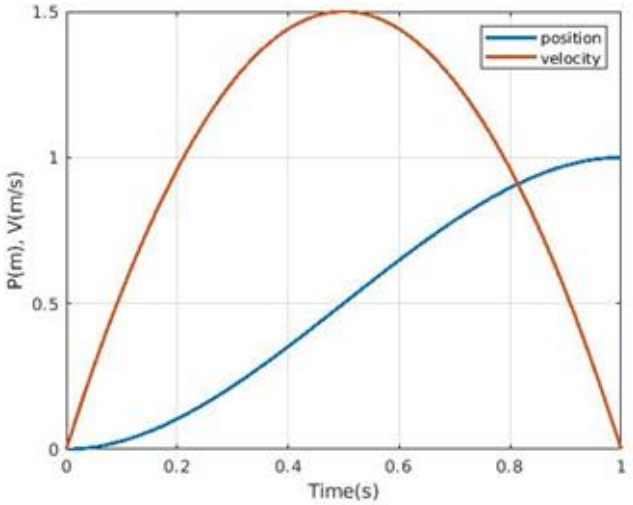
$$x = f(t) = at^3 + bt^2 + ct + d$$

$$\text{s.t.}\ f(0) = x_{\text{ini}}$$

$$f(1) = x_{\text{target}}$$

$$v(0) = 0$$

$$v(1) = 0$$



Multi-Segment 1D Trajectory

Passing Through an Intermediate Waypoint

What if we want the trajectory to **pass through an intermediate waypoint** x_{middle} at $t = 0.5$?

Solution:

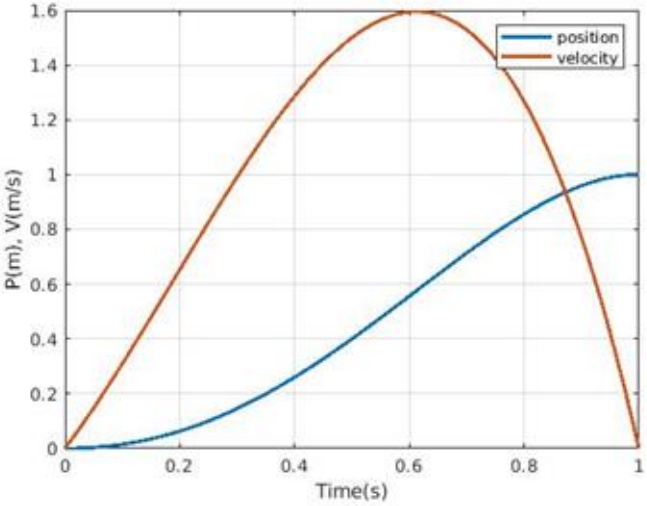
Increase the order of the polynomial to introduce more degrees of freedom

(more coefficients → more constraints satisfied)

Quartic Polynomial Representation:

$$x = f(t) = a\,t^4 + b\,t^3 + c\,t^2 + d\,t + e$$

$$\text{s.t. } f(0) = x_{\text{ini}}$$
$$f(1) = x_{\text{target}}$$
$$f(0.5) = x_{\text{middle}}$$
$$v(0) = 0$$
$$v(1) = 0$$



© Copyright Unmanned Systems Research Group

Limits of Polynomial Trajectory

Why Not Use Very High-Order Polynomials?

Generally, the more complex the trajectory, the more variables we need, and the higher the order the polynomial needs to be.

Numerical Instability at High Orders

High-order polynomials (e.g., 20th order) are prone to numerical instability due to the extreme scaling of powers:

$$f(t) = at^{20} + bt^{19} + \dots$$
$$f(0.1) = a0.1^{20} + b0.1^{19} + \dots + u$$

Even small numerical errors can drastically affect the trajectory due to the limited accuracy that can be achieved in a digital computer.

© Copyright Unmanned Systems Research Group



Outline

- Introduction
- Differential Flatness
- Polynomial Trajectory
- Minimum Snap Trajectory Generation
- Closed form Solution to Minimum Snap
- Implementation Details

© Copyright Unmanned Systems Research Group

General Problem Formulation

To minimize higher-order derivatives of flat outputs to achieve smooth and efficient motion.

- Minimize jerk: minimize angular velocity, good for visual tracking
- Minimize snap: minimize differential thrust, saves energy

Derivative (s)	Translation	Rotation	Input
1	Velocity		
2	Acceleration	Rotation	f
3	Jerk	Angular Velocity	
4	Snap	Angular Acceleration	$\boldsymbol{\tau}$

"We would like to design efficient trajectories, so what we actually care about is minimizing control effort, but we do this *indirectly* by minimizing snap."

© Copyright Unmanned Systems Research Group

Minimum Snap Trajectory Generation

Multi-segment minimum snap trajectory

Each segment is represented by a polynomial.

Formulation of multi-segments using piecewise polynomials:

$$f(t) = \begin{cases} f_1(t) = \sum_{i=0}^{N} p_{1,i}(t - T_0)^i & T_0 \le t \le T_1 \\ f_2(t) = \sum_{i=0}^{N} p_{2,i}(t - T_1)^i & T_1 \le t \le T_2 \\ \vdots & \vdots \\ f_M(t) = \sum_{i=0}^{N} p_{M,i}(t - T_M)^i & T_{M-1} \le t \le T_M \end{cases}$$

- It is not necessary to fix the polynomial order N, but using the same order across all segments simplifies the problem.
- **The time duration for each segment must be specified.**

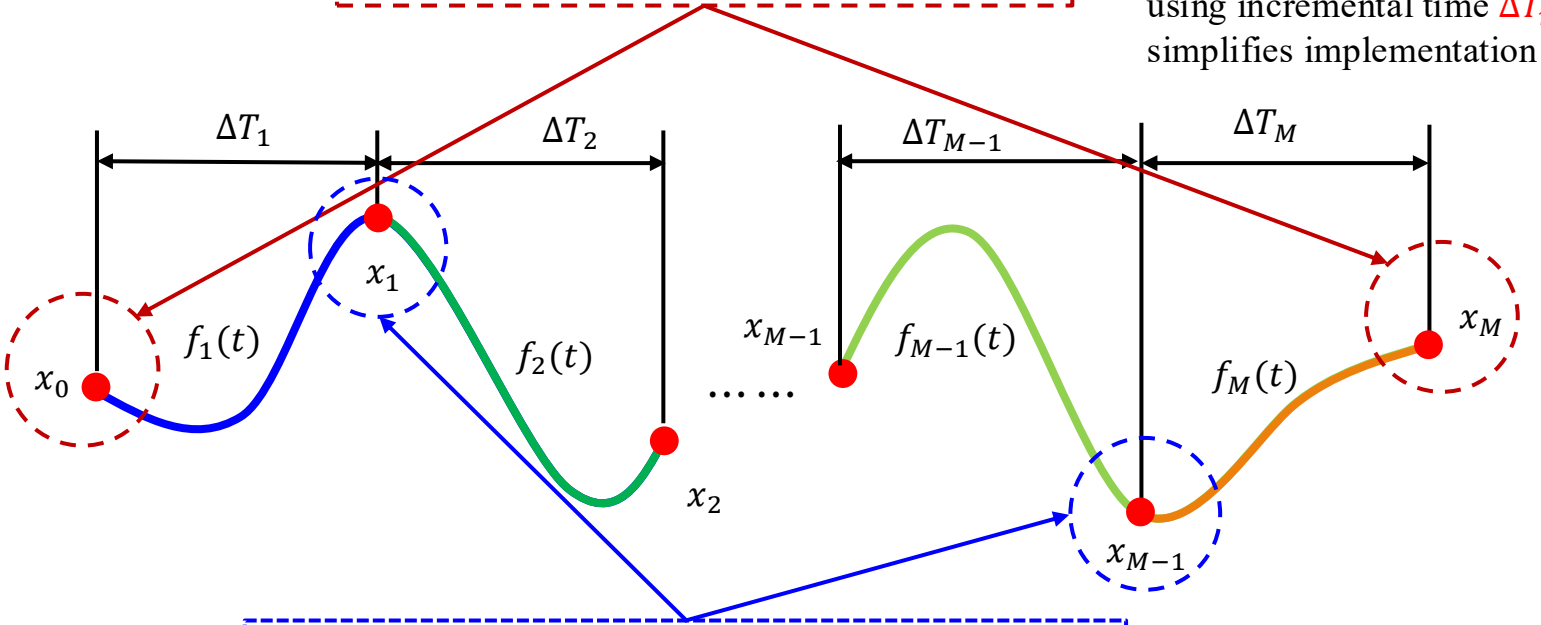
© Copyright Unmanned Systems Research Group

Minimum Snap Trajectory Generation

Constraints:

Boundary constraints: $\begin{cases} f_1^{(k)}(T_0) = x_0^{(k)} \\ f_M^{(k)}(T_M) = x_M^{(k)} \end{cases}$

Note: in trajectory generation, using absolute time T_m simplifies mathematical formulation, while using incremental time ΔT_m simplifies implementation in code.



Continuity Constraints: $f_m^{(k)}(T_m) = f_{m+1}^{(k)}(T_m)$

© Copyright Unmanned Systems Research Group

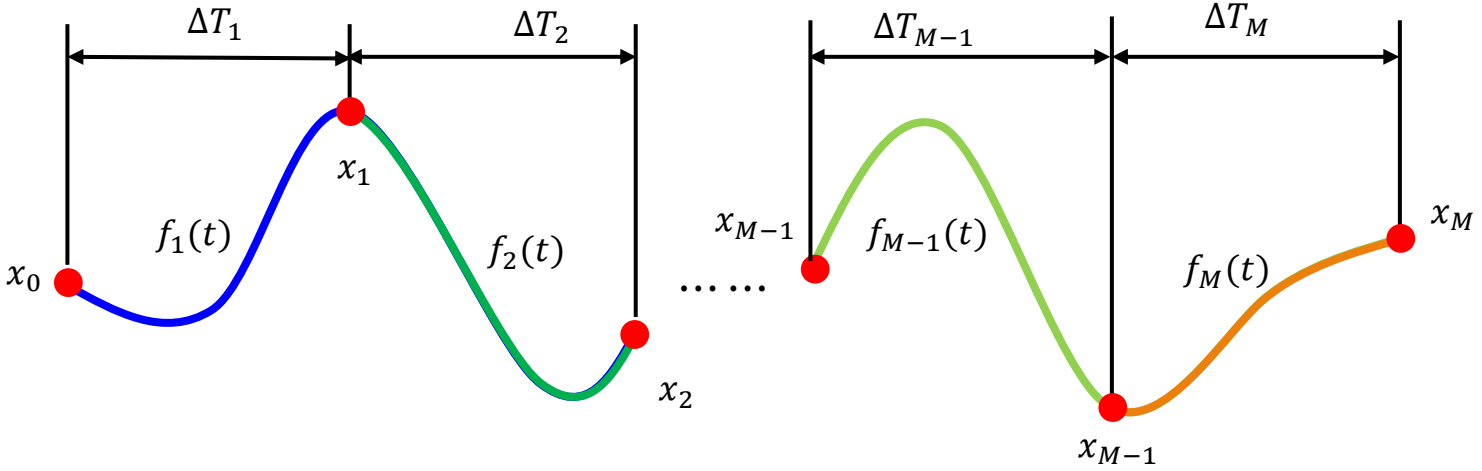
Minimum Snap Trajectory Generation

How to determine the trajectory order?

- Ensure smoothness at an order.
- Ensure continuity at an order.
- Minimize control input at an order.

Smoothness means its derivative is continuous!

How to minimize control input?



© Copyright Unmanned Systems Research Group

Minimum Snap Trajectory Generation

How to determine the trajectory order?

When designing a smooth trajectory, we need enough polynomial degree (=coefficients − 1) to satisfy all boundary and continuity constraints.

➢ One-segment Trajectory:

- Minimum jerk : $N = 2\times3\ - 1 = 5$ (3 initial conditions and 3 final conditions)
- Minimum snap: $N = 2\times4\ - 1 = 7$ (4 initial conditions and 4 final conditions)

➢Multi-Segment Trajectory (k-segments):

We want to obtain a minimum r-th derivative trajectory.

- Initial conditions $= r$
- Internal junctions $= r(k-1)$
- Final conditions $= r$

Total constrains $= r(k-1) + 2r$

Unknown coefficients: $(N+1)\times k \geq r(k-1) + 2r$

© Copyright Unmanned Systems Research Group

Minimum Snap Trajectory Generation

Cost function for one polynomial segment:

$$f(t) = \sum_i p_i t^i$$

$$\Rightarrow f^4(t) = \sum_{i\geq 4} i(i-1)(i-2)(i-3)\, t^{i-4} p_i$$

$$\Rightarrow \left(f^4(t)\right)^2 = \sum_{i\geq 4, j\geq 4} i(i-1)(i-2)(i-3)j(j-1)(j-2)\, t^{i+j-8} p_i p_j$$

$$\Rightarrow J(T) = \int_0^T \left(f^4(t)\right)^2 dt = \sum_{i\geq 4, j\geq 4} \frac{i(i-1)(i-2)(i-3)j(j-1)(j-2)}{i+j-7} T^{i+j-7} p_i p_j$$

$$\Rightarrow J(T) = \int_0^T \left(f^4(t)\right)^2 dt = \begin{bmatrix} \vdots \\ p_i \\ \vdots \end{bmatrix}^T \underbrace{\begin{bmatrix} \cdots & \sum_{i\geq 4, j\geq 4} \frac{i(i-1)(i-2)(i-3)j(j-1)(j-2)}{i+j-7} T^{i+j-7} & \cdots \end{bmatrix}}_{\mathbf{Q}_m} \begin{bmatrix} \vdots \\ p_j \\ \vdots \end{bmatrix}$$

$$\Rightarrow J_m(T) = \mathbf{p}_m^T \mathbf{Q}_m \mathbf{p}_m$$

Minimize this cost!

$$[\mathbf{Q}_m]_{i,j} = \begin{cases} \frac{i!\, j!}{(i-4)!\,(j-4)!} \cdot \frac{T_m^{i+j-7}}{i+j-7}, & if\ i,j \geq 4 \\ 0, & \text{otherwise} \end{cases}$$

© Copyright Unmanned Systems Research Group



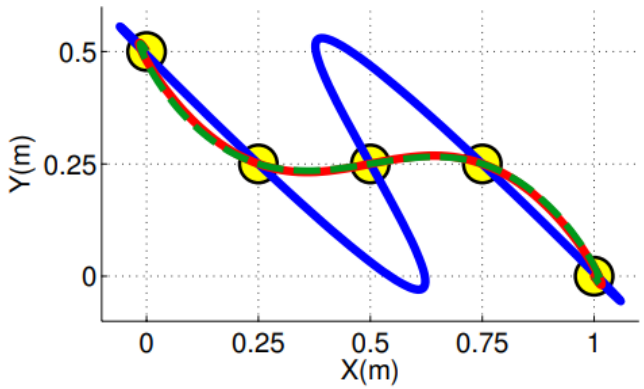
Outline

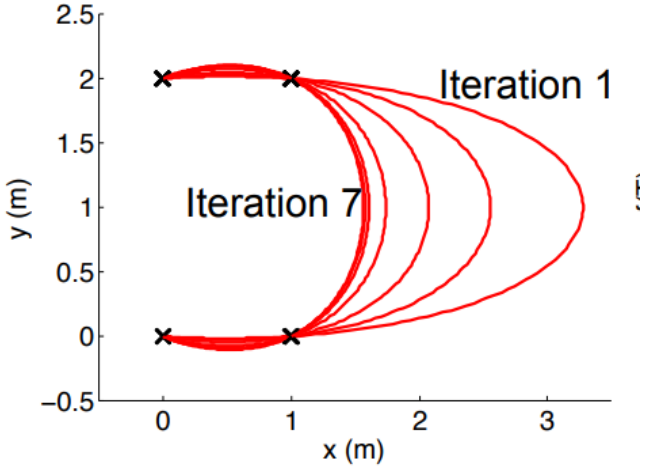
- Introduction
- Differential Flatness
- Polynomial Trajectory
- Minimum Snap Trajectory Generation
- Implementation Details
- Corrido-based Trajectory Optimization (Optional)

© Copyright Unmanned Systems Research Group

Time Allocation

- Piecewise trajectory depends on a piecewise time allocation.
- Time allocation significantly affect the final trajectory.
- How to get a proper time allocation?

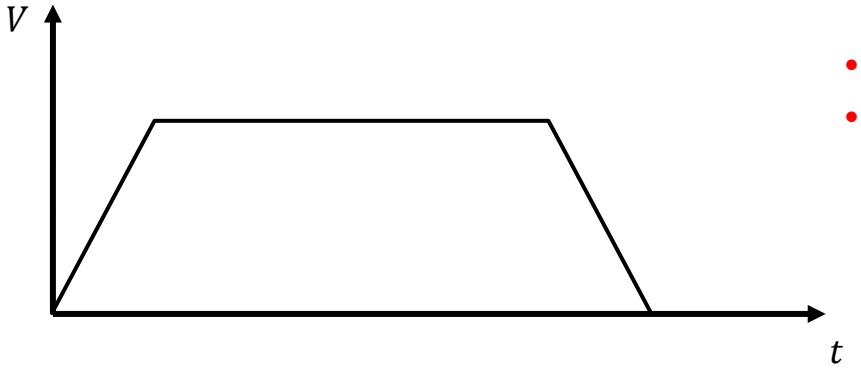




© Copyright Unmanned Systems Research Group

Time Allocation

- Use “trapezoidal velocity” time profile to get time durations for each pieces.
  - Assume in every piece, accelerate to max. velocity --> de accelerate to 0.
  - Accelerate + max. velocity + de accelerate.
- Use expected average velocity to get time durations for each pieces.



- Far away from optimal.
- Only get conservative time profile. (Why?)

Online generation of collision free trajectories for quadrotor flight in unknown cluttered environments , J. Chen, ICRA 2016
Fast, dynamic trajectory planning for a dynamically stable mobile robot, M. Shomin, IROS 2014

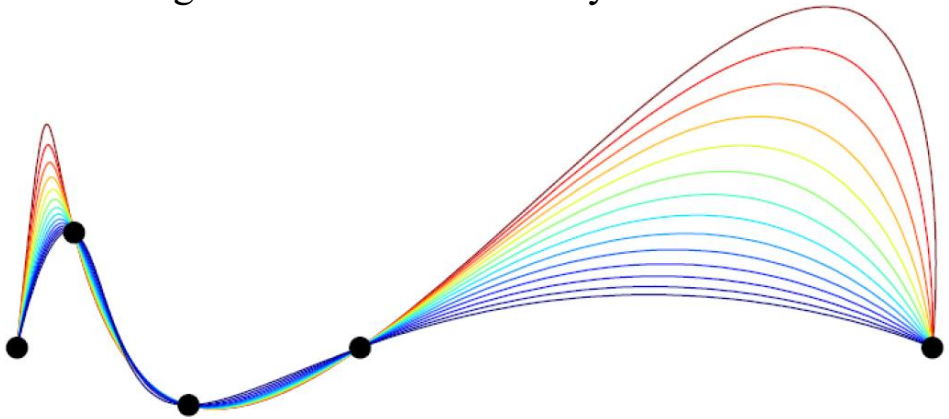
© Copyright Unmanned Systems Research Group

Iterative Numerical Solution

$$J_T = \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} \boldsymbol{Q}_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \boldsymbol{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} + k_T \sum_{i=1}^{M} T_i$$

Penalize time duration in the overall cost. Why?

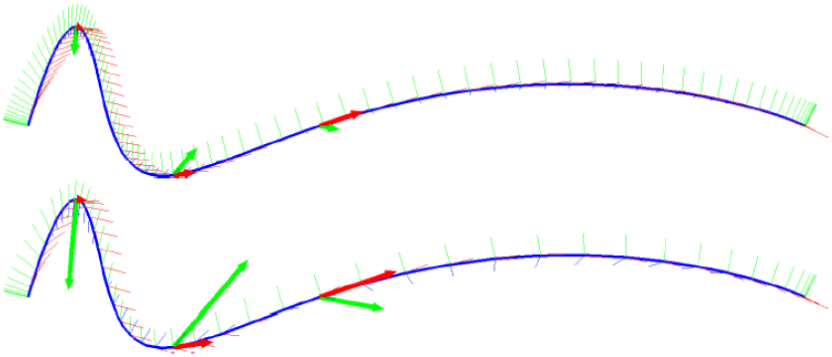
- Minimize this objective function **J**
- Get the gradient to T numerically

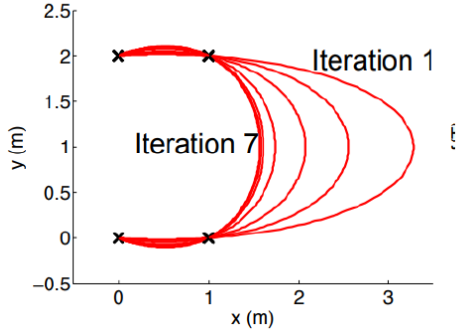


Richter C, BryA, Roy N. Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments[M]//Robotics research. Springer, Cham, 2016: 649-666.

© Copyright Unmanned Systems Research Group

Iterative Numerical Solution





- Varying segment time ratio
- Varying total time

Dynamic limit?

- Unconstrained, find the best time allocation, fix the ratio
- Scale the total trajectory time preserving the optimal ratio
- Until a constraint becomes active

- Varying segment time ratio again
- Fixed total time

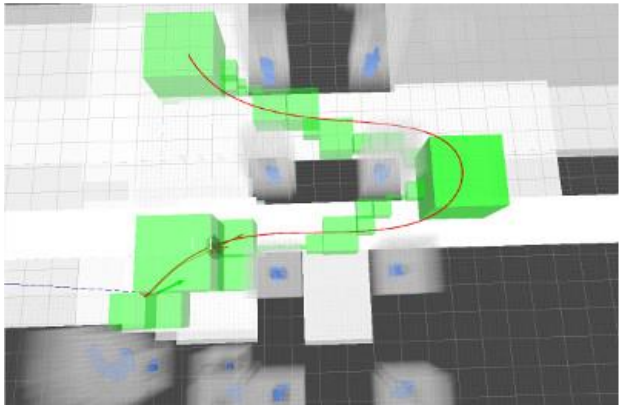
© Copyright Unmanned Systems Research Group

Corridor-based Trajectory Optimization

Motivation

What can we do if there are obstacles in the environment?

In UAV corridor-based trajectory planning, the UAV's **smooth trajectory is divided into segments**. Each segment lies inside a **corridor grid** (a safe 3D box). When transitioning from one grid to the next, the UAV must move **within the overlapping region** between the two adjacent corridor grids to ensure **collision-free motion**.



Online generation of collision free trajectories for quadrotor flight in unknown cluttered environments ,

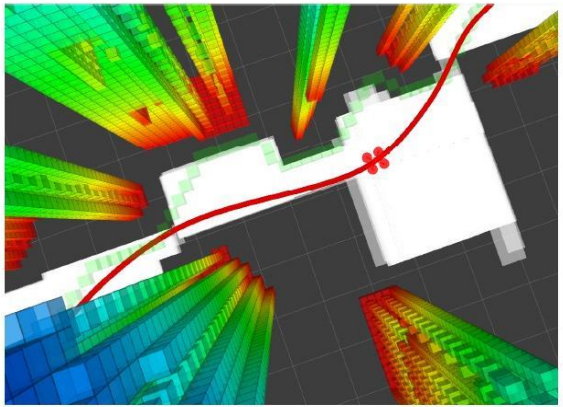
© Copyright Unmanned Systems Research Group

Corridor-based Trajectory Optimization

Connectivity Graph for Corridor Search

We use the standard A^* algorithm for initialization of a flight corridor. This requires maintaining a connectivity graph G for all free-space grids in addition to the octree-based map.

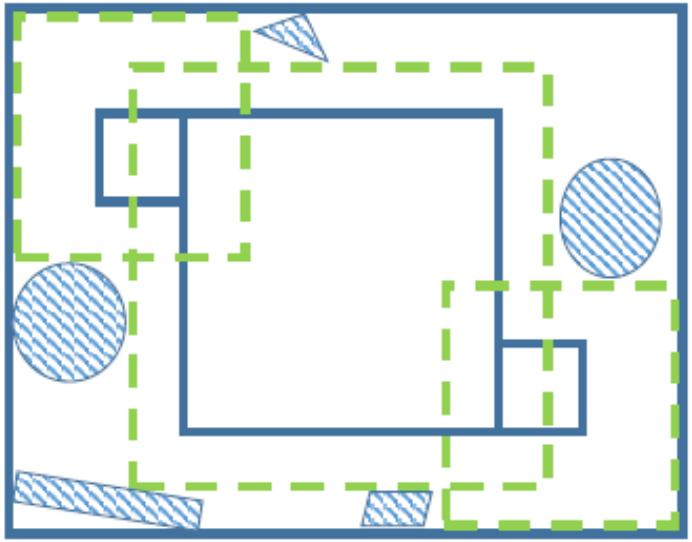




© Copyright Unmanned Systems Research Group

Corridor-based Trajectory Optimization

Maximal Volume Flight Corridor Generation



Corridor: A series of connected, convex, obstacle-free geometric volumes (typically cubes or convex polyhedra in 3D) that stretch from the start to the goal.

© Copyright Unmanned Systems Research Group

Corridor-based Trajectory Optimization

Polynomial Trajectory Generation

$$f(t) = \begin{cases} f_1(t) = \sum_{i=0}^{N} p_{1,i}(t - T_0)^i & T_0 \le t \le T_1 \\ f_2(t) = \sum_{i=0}^{N} p_{2,i}(t - T_1)^i & T_1 \le t \le T_2 \\ \vdots & \vdots \\ f_M(t) = \sum_{i=0}^{N} p_{M,i}(t - T_M)^i & T_{M-1} \le t \le T_M \end{cases}$$

Cost function

$$J(T) = \int_0^T \left(f^4(t)\right)^2 dt$$

- Boundary constraints
- Continuity constraints
- Safety constraints

$$\min \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} \boldsymbol{Q}_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \boldsymbol{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}$$

$$\text{s.t. } \mathbf{A}_{\text{eq}} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} = \mathbf{b}_{\text{eq}}$$

$$\mathbf{A}_{\text{ieq}} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} \le \mathbf{b}_{\text{ieq}}$$

Quadratic Program

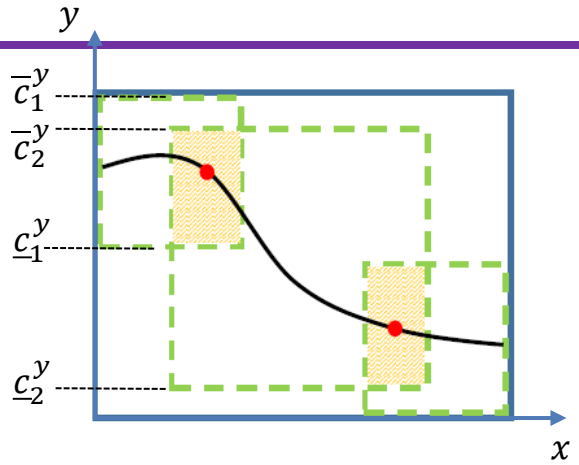
Chen, Jing et al. "Online generation of collision-free trajectories for quadrotor flight in unknown cluttered environments." 2016 IEEE International Conference on Robotics and Automation (ICRA) (2016): 1476-1483.

© Copyright Unmanned Systems Research Group

Corridor-based Trajectory Optimization

Enforcing Corridor Constraints

$$\begin{cases} \max(\underline{c}_i^l, \underline{c}_{i+1}^l) \le f_i(t_i) \le \min(\overline{c}_i^l, \overline{c}_{i+1}^l) \\ f_{i+1}(t_i) = f_i(t_i) \end{cases}$$



- c_i: the ith grid in corridor
- $f_i(t_i)$: the trajectory segment i evaluated at transition time (t_i)
- $\underline{c}_i^l, \overline{c}_i^l$: the lower and upper bounds of grid i along $l \in \{x, y, z\}$

Ensures continuity and enforces the UAV's position at grid transitions to lie within overlapping corridors.

The notation has been slightly adapted from the original paper to make it easier to understand in class.

Chen, Jing et al. "Online generation of collision-free trajectories for quadrotor flight in unknown cluttered environments." 2016 IEEE International Conference on Robotics and Automation (ICRA) (2016): 1476-1483.

© Copyright Unmanned Systems Research Group

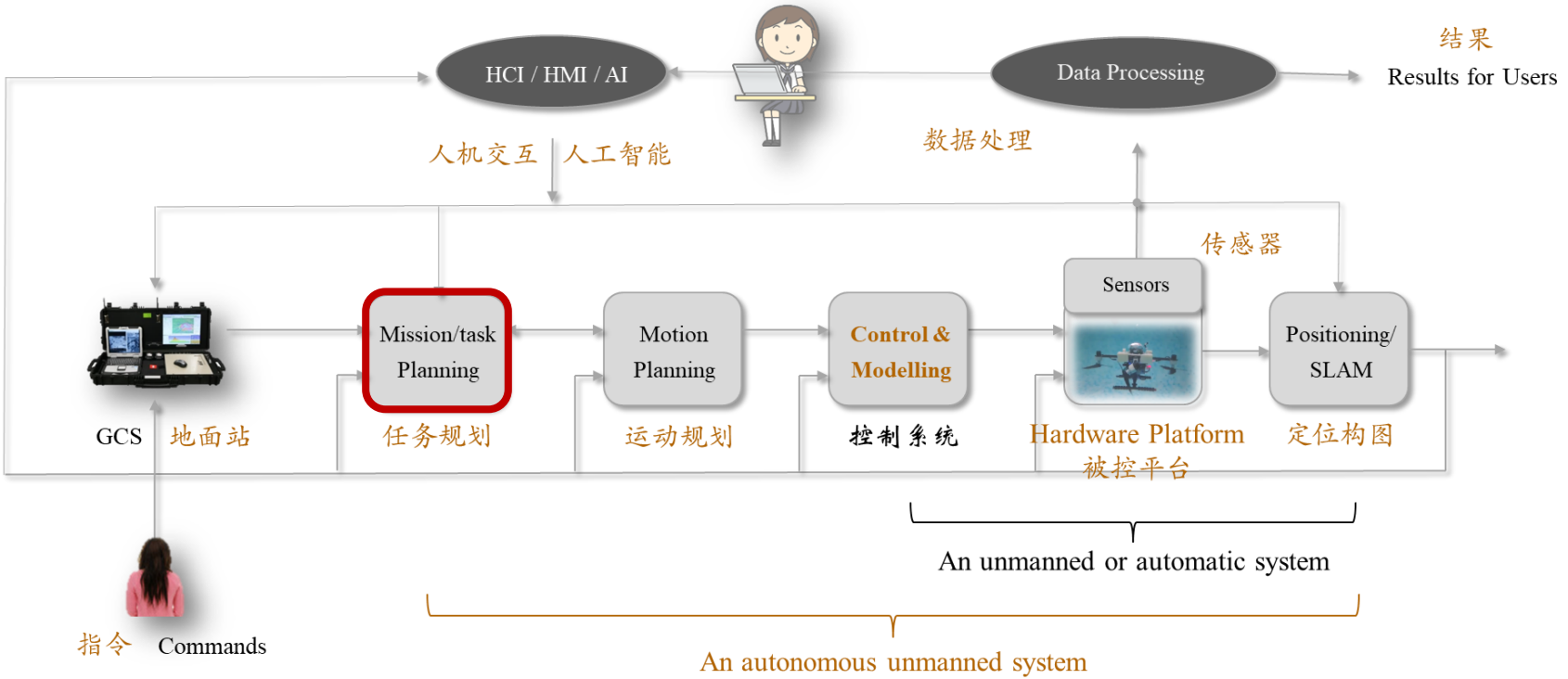

Full stop at each waypoint

Summary

- **Purpose:**
  - Enables safe, feasible, and efficient flight paths for UAV autonomy.
  - Bridges high-level planning & low-level control.
- **Differential Flatness:**
  - UAV trajectories can be planned in (p_x, p_y, p_z, ψ) for simplified control.
- **Minimum Snap Trajectories:**
  - Minimizes higher-order derivatives (snap) for smooth, energy-efficient flight.
  - Solved via constrained quadratic programming.
- **Time Allocation:**
  - Impacts feasibility; can use fixed, average, or optimized durations.
- **Corridor-based Planning:**
  - Ensures collision-free paths in cluttered environments.

© Copyright Unmanned Systems Research Group

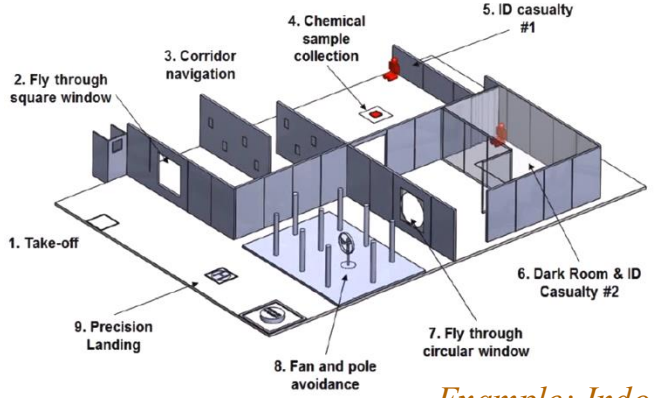
Task/Mission Planning

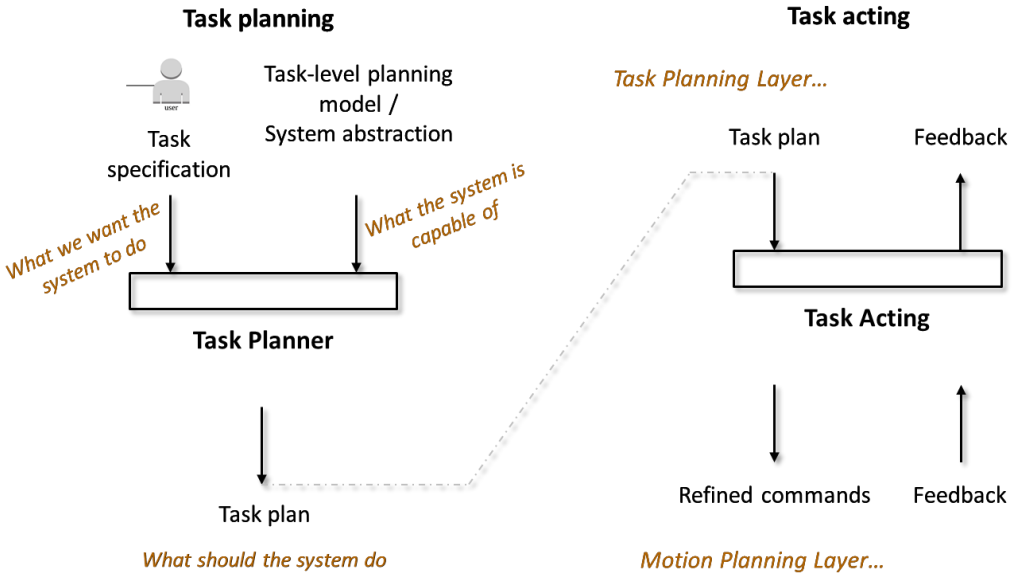


© Copyright Unmanned Systems Research Group

Task planning

- **Task planning** is to translate a mission into operations that may allow the successful execution of the mission, involving the planning of sensory operations, gross-motion planning with collision avoidance, and fine-motion planning with specific tasks.



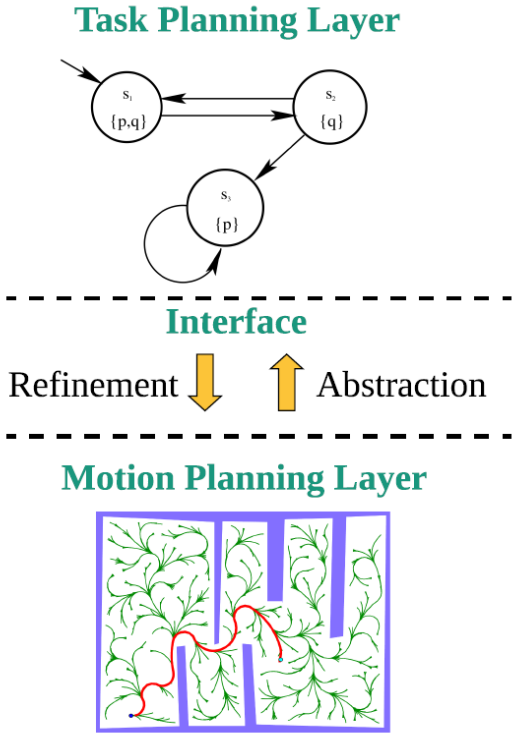


Example: Indoor Search & Rescue

© Copyright Unmanned Systems Research Group

From task planning to motion planning

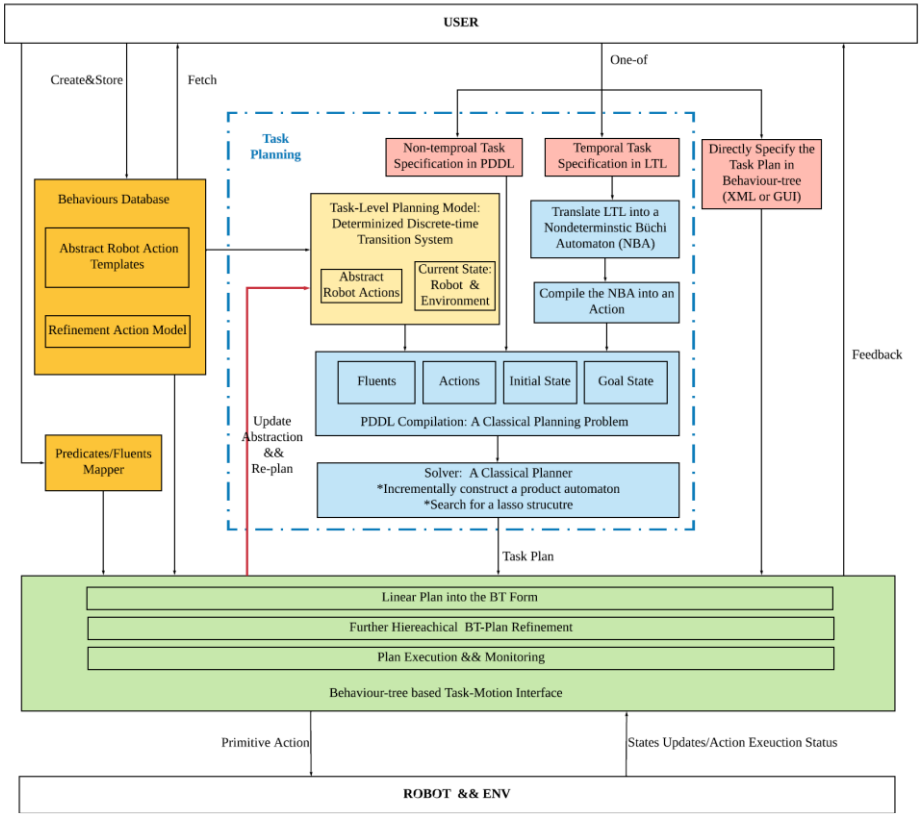
- Traditional one-way task-motion planning framework as shown on the right might not work for complicated problems.
- Task plan might not be implementable by the motion planner, because of over-simplified task planning model.
- The use of an independent interface to communicate between two planning layers might cause problems when dealing with complicated situations.



A Simplified Framework

© Copyright Unmanned Systems Research Group

Task planning and acting framework

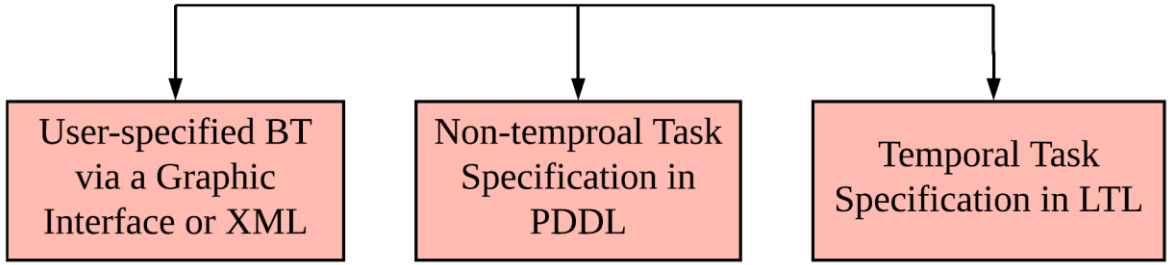


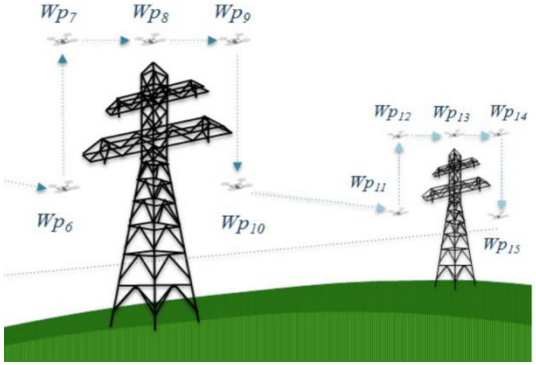
Motion planning

- **Pink: Task specifications**
- **Yellow: Model-related modules, e.g., behavior database, system abstraction, predicate mapper**
- **Blue: Classical planner based task planning algorithm**
- **Green: Behavior-tree based task acting algorithm**

© Copyright Unmanned Systems Research Group

Pink module: task specifications





Example: Powerline Inspection

Main functionality: **To allow users to properly specify the task requirements…**

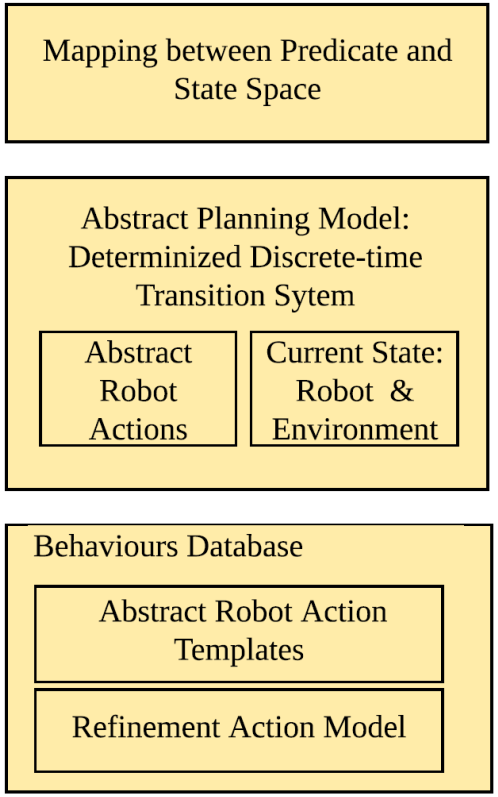
- Directly specify the task plan in the **behavior-tree** form (i.e., without going through the planner).
- Specify the non-temporal tasks using **Planning Domain Definition Language** (PDDL).
- Specify the task using **linear temporal logic** (LTL), which is suitable for tasks involving temporal constraints.

© Copyright Unmanned Systems Research Group

Yellow module: The model-related blocks

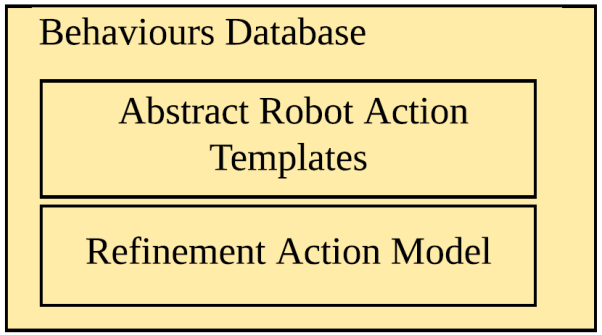
Main functionality: To define the task-level planning model…

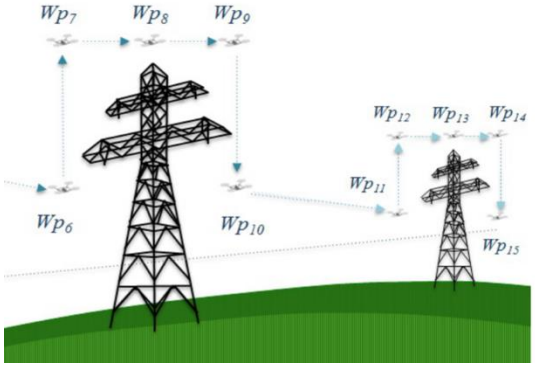
- Predicate abstraction is used to abstract the original state space. Each predicate is mapped to a subset of state space. The abstract state then can be described by these predicates.
- The abstract action manipulates the discrete states by changing the value of the predicate. Common abstract action template can be stored in behaviors database.
- The abstract system can be represented as a discrete-time transition system such as a state machine.



© Copyright Unmanned Systems Research Group

Yellow module: behaviors database





Main functionality: The behaviors database stores the planning models for common behavior of the unmanned system…

- **Primitive behavior:** Something can be directly executed by existing lower-level motions.
- **Composite behavior:** Each has a corresponding refinement model (which shows how to refine the composite behavior into a set of primitive behavior).
- Each behavior is abstracted using an **action template**.

© Copyright Unmanned Systems Research Group

Blue & green module: task planning & acting

Main functionality: To find a plan satisfying the task requirements…

- To compile the LTL synthesis problem into a classical planning problem (see, e.g., Patrizi *et al*., 2011)
- To solve the classical planning problem via any classical planner, such as
  - Fast Forward (FF) planner (see, e.g., Hoffmann, 2001)
  - LAMA planner (see, e.g., Richter and Westphal, 2010)

Main functionality: To communicate between task layer and motion layer…

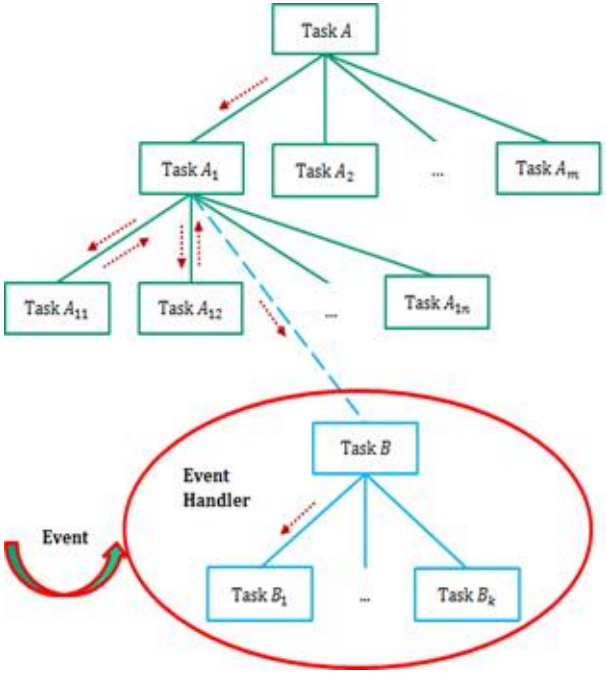
We use **behavior trees** as the main task-motion planning interface:

- Translate the linear plan into the behavior-tree based form;
- Further refine the tree-plan hierarchically using refinement model;
- Execute and monitor the plan; and
- Update the system abstraction and trigger the re-planning process if necessary.

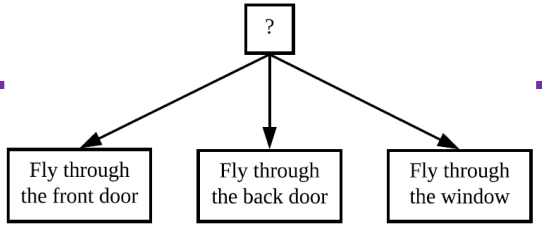
© Copyright Unmanned Systems Research Group

Behavior tree

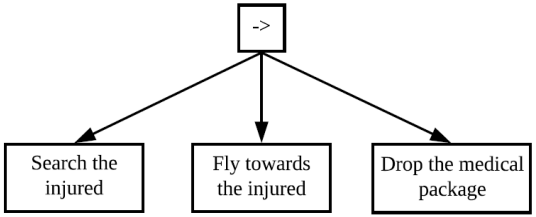
In a behavior tree, the tasks are organized into a tree and executed in a manner of depth-first traversal. Each leaf node task (action/condition node) contains only one single action. In other words, executing that leaf node task is equivalent to executing the corresponding action.



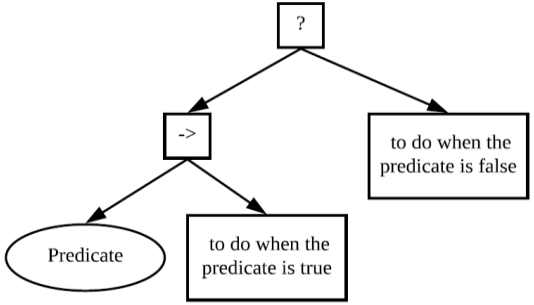
Example: A behavior tree



Example: Selector node



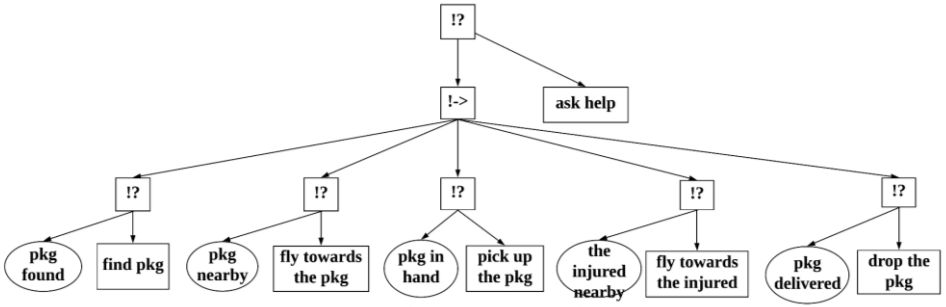
Example: Sequence node



Example: If–else-then node

© Copyright Unmanned Systems Research Group

Behaviour tree vs finite state machine



Behavior tree

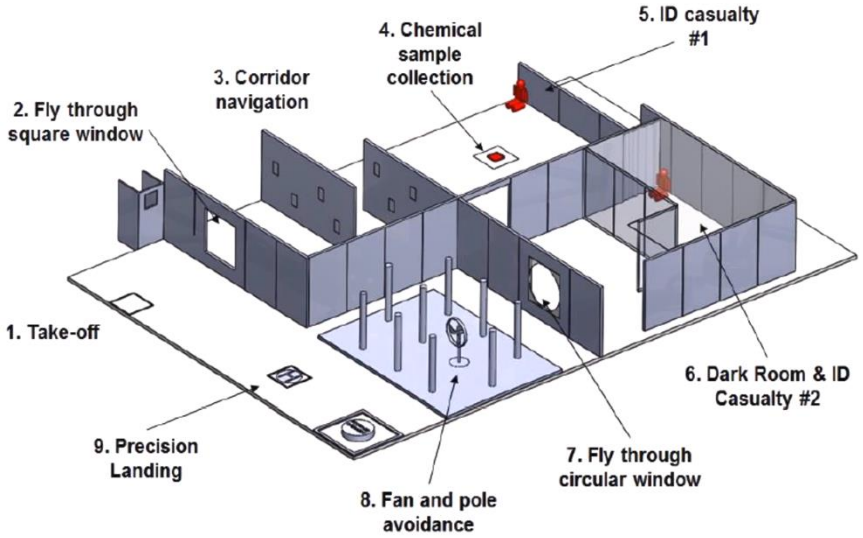
- Both approaches can be translated to machine code in finite time.
- However, the hierarchical structure of a behaviour tree allows easy selection, grouping and management of submodules.

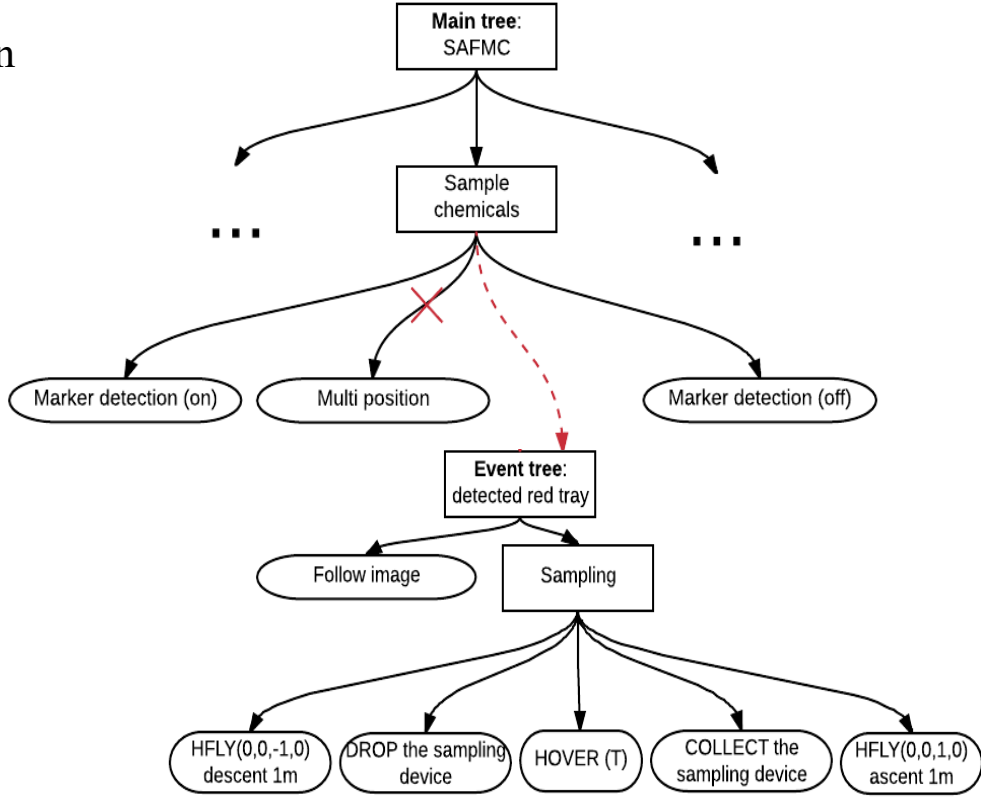


Finite state machine

© Copyright Unmanned Systems Research Group

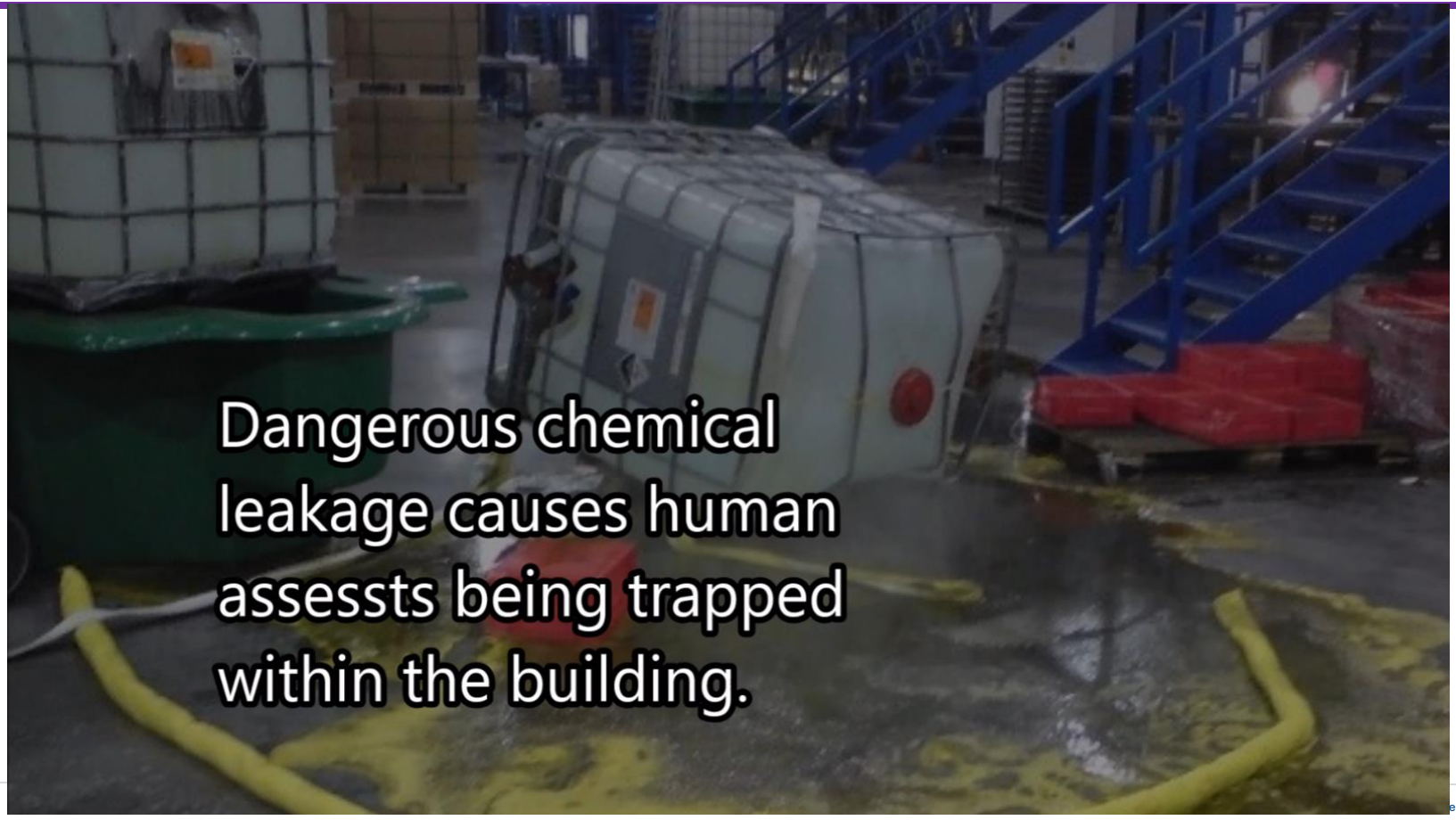
Example: Behavior tree of an autonomous unmanned aerial system for search and rescue in indoor chemical disaster…





© Copyright Unmanned Systems Research Group

UAS for indoor search & rescue





Thank You!

© Copyright Unmanned Systems Research Group